Implementing the VAE

Problem

Not enough data to train all of the different models that we want to create, so we want to create data augmentation methods to make enough data to work out. This method is a lot simpler than using a GAN, and likely a lot more stable, so use the VAE before using more difficult methods.

- Data scarcity
- Unstable solution space

Implementation Concerns

- Expensive to compute - lots or GPU memory required
- Checkerboard artifacts, depending on how the sampling works, you could sample one chunk to be different from the adjacent chunk, leading to checkerboarding
    - `torch.nn.functional.interpolate` configured with nearest mode, thus removing/reducing checkerboarding as the difference between chunks is "smoothed" out

Large Tasks to Complete

- Create a model to generate the "latent space"
    - Looks at the features, learns what values are likely depending on what's around it
    - Create a probability distribution (which we something to get trained on)
- Sample from the probability distribution depending on the values inputted into the model and it should come up with an augmented data cube

Implementation Details

- **Kullback-Leibler Divergence**
    - Same loss as the GAN, measures the difference between the ground truth and the produced version, not sure why its KL

$$\mathcal{L}_{VAE} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta \cdot D_{\mathrm{KL}}(q_\phi(z|x)||p(z))$$

    - Similar to Earth Mover's Distance
    - Cannot rely exclusively on MSE, as it cannot preserve self-attention across hyperspectral bands
- **Leaky ReLU**
    - Preventing dead neurons while preventing non-linearity (`nn.InstanceNorm3d` and `nn.GroupNorm`)
- **Reparameterization**

$$z = \mu + \sigma \times \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

    - Sample from the latent space
- Need to implement many different tests to make sure that it is working along the way because it is very unstable.
    - Create tests for the images that come out, make scripts to automate them so we can check many at a time before we continue training: **Spectral Angle Mapper (SAM) loss** and **Spectral Information Divergence (SID)** loss (no idea what these are but these are what we need to use to evaluate model)
    - This is already implemented somewhere so we can just steal their code, and use it to evaluate model performance as we go
    - The loss function should be a combination of the two above functions
- Possibility of "posterior collapse", when the decoder ignores the latent vector z and just creates generic outputs (basically, it learns what the loss function wants, and only does that isntead of listening to the encoder, leading to good output, but always the same, so useless for data augmentation)
  -Something called KL annealing: First clamp β to 0, allowing the encoder and decoder to minimize the loss first, then increase the β parameter, to further organize latent space after reconstruction capacity has stabilized

Implementation Steps

1. Normal image trasnformation augmentation methods
    1. Sliding window, rotation, amplitude
    2. Cannot have the window that is too large or too small, or the model won't be able to learn the connections, or it will have too many parameters (and it

will be seeing the same data too much because of data sparsity, and the overlaps) ~ about 15x15x15 data cubes?
    1. Normalization of the data to hopefully get stable training
2. PyTorch nn.Conv3d Modules `(Batch_Size, Channels, Depth, Height, Width)`
    1. Starting variable definitions (32/64, 1, 200, 9, 9) (no clue if these are good values, but these are good values to start with)
    2. The variables definitions depend on what affects the spectral value -> do the surrounding landscape data affect the spectral data (nxmx1)? or is it only the surrounding spectral data that matters (1x1x200)? (its definitely somewhere in between but that's what we need to figure out)
    3. The loss function is based on SAD and SID (there's a little chunk a couple paragraphs above this)

Encoder Model Implementation

Input data: `(Batch_Size, Channels, Depth, Height, Width)`
Output data: Returns a randomly sample from the normal distribution $z \sim \mathcal{N}(\mu, \sigma^2)$

Encoder Stage	Operational Module Sequence	Kernel Details	Stride	Output Channels
Input Data	`Formatted 5D PyTorch Tensor`	—	—	—
Convolution Block 1	`Conv3d` → `InstanceNorm3d` → `LeakyReLU`	(3, 3, 3)	2	16
Convolution Block 2	`Conv3d` → `InstanceNorm3d` → `LeakyReLU`	(3, 3, 3)	2	32
Convolution Block 3	`Conv3d` → `InstanceNorm3d` → `LeakyReLU`	(3, 3, 3)	2	64
Flattening	`nn.Flatten(start_dim=1)`	—	—	1D Vector

Comes out with a flattened one-dimensional vector. To distinguish from a deterministic decoder, it doesn't return the mean vector μ, instead, it provides a vector z that is randomly sampled from the probability distribution defined by the encoder's outputs: $z \sim \mathcal{N}(\mu, \sigma^2)$ (vector z randomly sampled from the normal distribution which has μ as the mean and σ as the standard deviation)

- The random sampling must be performed after the decoder has been trained, as adding randomness will add instability to the backpropagation (unless we use non-gradient dependent methods of optimization)

Decoder Model Implementation

Decoder Stage	Artifact-Free Upsampling Module Sequence	Spatial-Spectral Scale	Output Channels
Latent Expansion	`nn.Linear` $\rightarrow$ `Tensor.view`	Base Volume	64
Upsampling Block 1	`Interpolate(mode='nearest')` $\rightarrow$ `Conv3d(stride=1)` $\rightarrow$ `InstanceNorm` $\rightarrow$ `LeakyReLU`	$2\times$	32
Upsampling Block 2	`Interpolate(mode='nearest')` $\rightarrow$ `Conv3d(stride=1)` $\rightarrow$ `InstanceNorm` $\rightarrow$ `LeakyReLU`	$2\times$	16
Final Generation	`Interpolate(mode='nearest')` $\rightarrow$ `Conv3d(stride=1)` $\rightarrow$ `Sigmoid`	$2\times$	1

- Using Nearest-neighbor interpolation to mitigate checkerboard artifacts caused by `ConvTranspose3d`
- Requires implementation only with pytorch tensor operations (so essentially, can't do anything to the data outside of the program if backpropagation is used)